

CriptoIA: Um Sistema de Análise de Criptomoedas Baseado em Indicadores Técnicos e Inteligência Artificial Multimodal

Gabriel Teixeira
Ciência da Computação
Universidade Franciscana
Santa Maria, Brasil
gabriel.teixeira@ufn.edu.br

Resumo—Este trabalho apresenta o CriptoIA, um sistema computacional desenvolvido em Python que integra coleta automatizada de dados financeiros, cálculo de indicadores de análise técnica e inteligência artificial multimodal para suporte à tomada de decisão no mercado de criptomoedas. O sistema conecta-se à API pública da Binance por meio da biblioteca CCXT para obter dados históricos de candles OHLCV (Open, High, Low, Close, Volume) e preços em tempo real. Sobre esses dados, calcula-se um conjunto de indicadores amplamente utilizados na análise técnica: Média Móvel Simples (SMA) de 20 e 50 períodos, Índice de Força Relativa (RSI-14), MACD (*Moving Average Convergence Divergence*) e Bandas de Bollinger. Todos os indicadores são implementados com a biblioteca Pandas, sem dependências externas de análise técnica. O gráfico técnico gerado em alta resolução é submetido, em conjunto com os valores numéricos dos indicadores, ao modelo *Gemini 2.5 Flash* do Google, que produz uma recomendação estruturada em JSON com ação (COMPRAR, VENDER ou MANTER), justificativa técnica detalhada, preço-alvo e estimativa de tempo. Os resultados são apresentados em um dashboard web interativo gerado localmente via Plotly, com tema escuro e micro-animações CSS. A arquitetura modular, a integração entre análise quantitativa clássica e IA generativa multimodal, e a baixa barreira de instalação distinguem o CriptoIA como uma ferramenta educacional e de pesquisa relevante no contexto de finanças computacionais.

Index Terms—Criptomoedas, Análise Técnica, Inteligência Artificial, Google Gemini, Bitcoin, MACD, RSI, Bandas de Bollinger, Python, CCXT, Plotly.

I. INTRODUÇÃO

O mercado de criptomoedas cresceu de forma expressiva ao longo da última década. O Bitcoin, lançado em 2009 por Satoshi Nakamoto [1], inaugurou um paradigma de ativos digitais descentralizados baseados em tecnologia *blockchain*, que hoje inclui milhares de criptomoedas negociadas em bolsas ao redor do mundo. Em 2024, a capitalização total do mercado ultrapassou dois trilhões de dólares, tornando-se um segmento relevante tanto para investidores individuais quanto para instituições financeiras [2]. No Brasil, o interesse por criptoativos também cresceu substancialmente, com a

Receita Federal registrando volumes recordes de declarações envolvendo ativos digitais [3].

Diferentemente dos mercados financeiros tradicionais, as bolsas de criptomoedas operam ininterruptamente, 24 horas por dia, sete dias por semana, sem pausas para fins de semana ou feriados. Essa característica, aliada à elevada volatilidade intrínseca dos ativos digitais, torna o acompanhamento manual contínuo praticamente inviável para a maioria dos investidores [4]. Nesse cenário, ferramentas de análise técnica automatizada surgem como recurso valioso para identificar padrões de mercado e oportunidades de negociação de forma sistemática.

A análise técnica é um método de avaliação de ativos financeiros fundamentado no estudo de dados históricos de preço e volume [5]. Por meio de indicadores matemáticos derivados das séries temporais de preços, o analista busca identificar tendências, momentum, condições de sobrecompra e sobrevenda, além de outros sinais relevantes para a tomada de decisão [6]. Dentre os indicadores mais utilizados destacam-se as Médias Móveis Simples e Exponenciais, o RSI (*Relative Strength Index*), o MACD e as Bandas de Bollinger.

Paralelamente, o surgimento de modelos de linguagem grandes (*Large Language Models* — LLMs) com capacidade multimodal abre novas possibilidades para a análise financeira computacional. Modelos como o Gemini [7] são capazes de processar simultaneamente texto e imagens, permitindo a interpretação de gráficos de preços de forma visual e incorporando padrões gráficos que são difíceis de capturar somente por métricas numéricas. A utilização de IA generativa como suporte à decisão financeira é um campo emergente com potencial significativo [8].

Neste contexto, o presente trabalho propõe e descreve o desenvolvimento do **CriptoIA**, um sistema que combina coleta automatizada de dados da Binance, cálculo de indicadores técnicos implementados em Pandas, geração de gráficos de alta qualidade e análise por IA multimodal em uma pipeline coesa. As contribuições específicas deste trabalho são:

- Integração entre análise técnica clássica e modelo de linguagem multimodal (Gemini 2.5 Flash) para reco-

mendação de operações em criptomoedas, com saída estruturada em JSON;

- Implementação de cinco indicadores técnicos utilizando exclusivamente Pandas, sem bibliotecas externas de análise técnica;
- Arquitetura modular em Python com separação clara entre pacotes *backend* e *frontend* e sem dependências circulares;
- Dashboard web interativo gerado localmente como HTML autossuficiente, sem necessidade de servidor;
- Prompt de engenharia com interpretações qualitativas automáticas dos indicadores, enriquecendo o contexto enviado ao modelo.

Este trabalho está organizado da seguinte forma: a Seção II apresenta o referencial teórico; a Seção III descreve a arquitetura do sistema; a Seção IV detalha a implementação de cada módulo; a Seção V discute os resultados obtidos; e a Seção VI apresenta as conclusões e trabalhos futuros.

II. REFERENCIAL TEÓRICO

A. Análise Técnica em Mercados Financeiros

A análise técnica parte do princípio de que toda a informação relevante para a avaliação de um ativo já está refletida em seu preço de mercado, e que os preços se movem em tendências identificáveis a partir de dados históricos [9]. Três premissas fundamentam essa abordagem: (i) o preço desconta tudo; (ii) os preços se movem em tendências; e (iii) a história se repete. No contexto brasileiro, a análise técnica é amplamente adotada por operadores de bolsa e investidores de varejo como instrumento de apoio à decisão, conforme documentado por [6].

A análise técnica complementa, e muitas vezes se contrapõe, à análise fundamentalista, que avalia o valor intrínseco de um ativo com base em dados econômicos, balanços e perspectivas de crescimento [4]. No mercado de criptomoedas, onde os fundamentos são mais difíceis de mensurar objetivamente, a análise técnica ganha protagonismo como ferramenta de curto e médio prazo.

B. Indicadores Técnicos Utilizados

1) *Média Móvel Simples (SMA)*: A Média Móvel Simples de n períodos é definida pela equação:

$$SMA_n(t) = \frac{1}{n} \sum_{i=0}^{n-1} P_{t-i} \quad (1)$$

onde P_t é o preço de fechamento no período t . A SMA suaviza as flutuações de curto prazo e evidencia a direção da tendência. No CriptoIA, são calculadas a SMA de 20 períodos (MM20, tendência de curto prazo) e a de 50 períodos (MM50, tendência de médio prazo). O cruzamento dessas médias é um dos sinais mais utilizados: quando MM20 cruza acima da MM50, configura-se o *golden cross*, indicador de possível início de tendência de alta; o cruzamento inverso, denominado *death cross*, sinaliza tendência de baixa.

2) *RSI — Índice de Força Relativa*: O RSI, proposto por J. Welles Wilder em 1978 [10], mede o momentum do preço em uma escala contínua de 0 a 100:

$$RSI = 100 - \frac{100}{1 + RS} \quad (2)$$

onde $RS = \bar{G}/\bar{L}$, sendo $\bar{G}$ a média dos ganhos e $\bar{L}$ a média das perdas nos últimos n períodos (convencionalmente $n = 14$). O RSI acima de 70 indica condição de sobrecompra, sugerindo possível reversão de baixa; abaixo de 30, indica sobrevenda e possível reversão de alta. No CriptoIA, o RSI é calculado via variação diária dos preços de fechamento, com ganhos e perdas separados pelo método `clip()`.

3) *MACD — Moving Average Convergence Divergence*: O MACD, desenvolvido por Gerald Appel na década de 1970, é calculado como a diferença entre a Média Móvel Exponencial (EMA) de 12 e de 26 períodos:

$$MACD = EMA_{12} - EMA_{26} \quad (3)$$

A EMA pondera os preços recentes de forma exponencialmente decrescente, tornando-se mais responsiva às variações recentes do que a SMA. A linha de sinal é a EMA de 9 períodos do próprio MACD, e o histograma representa a diferença entre MACD e linha de sinal. Cruzamentos ascendentes da linha MACD sobre a linha de sinal são interpretados como sinais de compra; cruzamentos descendentes, como sinais de venda.

4) *Bandas de Bollinger*: Desenvolvidas por John Bollinger [11], as Bandas de Bollinger consistem em uma banda central (SMA de n períodos) e duas bandas externas posicionadas a k desvios padrão:

$$BB^+ = SMA_n + k \cdot \sigma_n \quad (4)$$

$$BB^- = SMA_n - k \cdot \sigma_n \quad (5)$$

onde σ_n é o desvio padrão dos preços de fechamento nos últimos n períodos. No CriptoIA, utiliza-se $n = 20$ e $k = 2$. As bandas se expandem em períodos de alta volatilidade e se contraem em períodos de baixa volatilidade (*squeeze*). Preços próximos à banda superior indicam sobrecompra relativa; próximos à inferior, sobrevenda.

C. Modelos de Linguagem com Capacidade Multimodal

Os LLMs com visão computacional (*Vision-Language Models*) recebem como entrada sequências de tokens tanto textuais quanto visuais, permitindo raciocinar sobre imagens e responder perguntas baseadas em conteúdo visual [12]. Essa capacidade os torna candidatos naturais para análise de gráficos financeiros, nos quais padrões visuais como formações de candlestick, divergências e breakouts são fundamentais.

O Gemini 2.5 Flash, modelo utilizado neste trabalho, pertence à família de modelos multimodais do Google DeepMind [7]. Trata-se de um modelo otimizado para baixa latência e alto volume de inferência, mantendo capacidade de raciocínio complexo sobre imagens e textos. Sua utilização via API

REST, com autenticação por chave de API e estrutura de prompts flexível, o torna acessível para integração em sistemas externos sem necessidade de infraestrutura especializada.

D. Mercado de Criptomoedas e Ferramentas de Acesso

A Binance é a maior *exchange* de criptomoedas do mundo em volume negociado [13]. Disponibiliza uma API REST pública que permite, sem autenticação, o acesso a dados históricos de candles OHLCV com até 1.000 registros por requisição, além de ticker de preços em tempo real. Cada candle OHLCV agrega cinco informações por período: preço de abertura, máximo, mínimo, fechamento e volume negociado.

A biblioteca CCXT (*CryptoCurrency eXchange Trading Library*) é uma solução de código aberto que unifica o acesso a mais de cem *exchanges* de criptomoedas, incluindo a Binance, por meio de uma interface padronizada [14]. Isso permite que o CriptoIA seja facilmente adaptado para operar com outras exchanges, substituindo apenas o identificador da exchange na inicialização do cliente CCXT.

III. ARQUITETURA DO SISTEMA

O CriptoIA foi projetado seguindo os princípios de separação de responsabilidades (*Separation of Concerns*) e modularidade, de modo que cada componente realiza uma função bem delimitada e pode ser substituído ou estendido independentemente. A Figura 1 apresenta o diagrama de componentes do sistema, e a Figura 2 ilustra o fluxo de execução da pipeline.

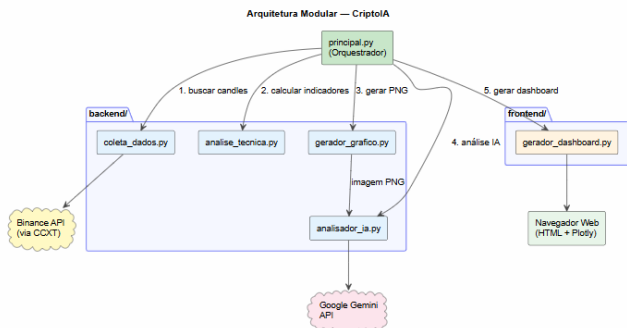


Figura 1. Diagrama de arquitetura modular do sistema CriptoIA.

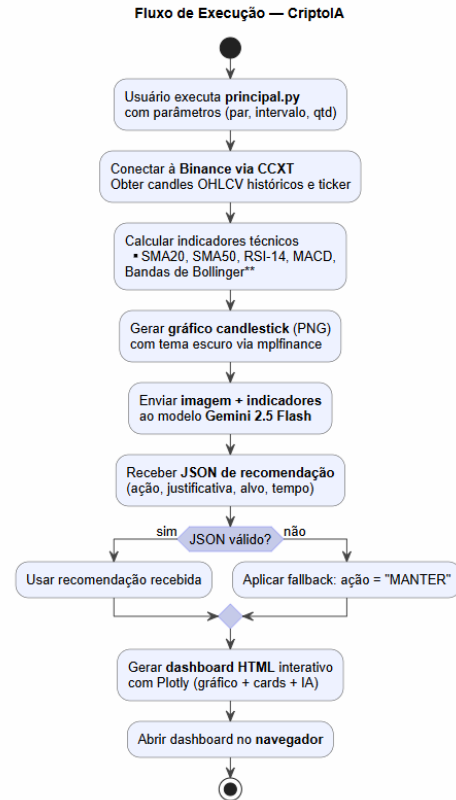


Figura 2. Fluxo de execução da pipeline do CriptoIA.

O sistema é organizado em dois pacotes Python principais:

- **backend/**: contém os quatro módulos responsáveis pela lógica de negócio, coleta de dados, análise técnica, geração de gráfico e comunicação com a IA. Nenhum módulo do backend importa outro módulo do mesmo pacote, eliminando dependências circulares e facilitando testes unitários isolados.
- **frontend/**: contém o módulo de geração do dashboard HTML interativo, que consome exclusivamente dados processados pelo backend, sem acesso direto à API da Binance ou ao Gemini.

O arquivo `principal.py` atua como orquestrador, inicializando e encadeando os módulos em uma pipeline sequencial de cinco etapas. A Tabela I lista as dependências externas do projeto com suas versões mínimas.

Tabela I
DEPENDÊNCIAS EXTERNAS DO CRIPTOIA

Biblioteca	Versão	Finalidade
ccxt	≥4.0	Acesso à API da Binance
pandas	≥2.0	Manipulação de dados tabulares
matplotlib	≥3.8	Gráficos estáticos (PNG)
mplfinance	≥0.12	Gráficos estilo financeiro
plotly	≥5.0	Gráficos interativos (HTML)
google-genai	≥1.0	Integração com Google Gemini
Pillow	≥10.0	Leitura de imagens (PNG)
python-dotenv	≥1.0	Gerenciamento de variáveis de ambiente

IV. IMPLEMENTAÇÃO DOS MÓDULOS

A. Módulo Orquestrador - *principal.py*

O ponto de entrada do sistema realiza três configurações iniciais antes de invocar a pipeline: (1) define a codificação UTF-8 para saída padrão, garantindo compatibilidade com o Windows; (2) adiciona o diretório raiz do projeto ao `sys.path`, viabilizando as importações dos subpacotes backend e frontend; e (3) instancia o parser de argumentos via `argparse`, expondo os parâmetros configuráveis em linha de comando:

- `-par`: par de negociação (padrão: BTC/USDT);
- `-intervalo`: timeframe dos candles — 1h, 4h, 1d ou 1w (padrão: 1d);
- `-quantidade`: número de candles a buscar (padrão: 90).

A função `executar_analise()` encadeia as cinco etapas da pipeline, conforme o Algoritmo 1:

Listing 1. Pipeline principal do sistema CriptoIA.

```
1 def executar_analise(par, intervalo, quantidade):
2     df_historico = buscar_dados_historicos(
3         par, intervalo, quantidade)
4     dados_ticker = buscar_preco_atual(par)
5
6     df_enriquecido = enriquecer_dataframe(df_historico)
7     indicadores = resumir_indicadores(df_enriquecido)
8
9     caminho_grafico = gerar_grafico_candlestick(
10         df_enriquecido, nome_arquivo="grafico.png")
11
12     analise_ia = analisar_mercado_com_gemini(
13         caminho_grafico, indicadores)
14
15     caminho_html = gerar_dashboard(
16         df=df_enriquecido,
17         dados_ticker=dados_ticker,
18         analise_ia=analise_ia,
19         indicadores=indicadores,
20         par=par)
21     webbrowser.open(f"file://{caminho_html}")
```

B. Módulo de Coleta de Dados — *coleta_dados.py*

Este módulo é responsável pela comunicação com a API pública da Binance utilizando a biblioteca CCXT. A função auxiliar `obter_cliente_binance()` inicializa o cliente com suporte opcional a autenticação: quando as variáveis de ambiente `BINANCE_API_KEY` e `BINANCE_SECRET_KEY` estão configuradas (carregadas via `python-dotenv`), o cliente opera no modo autenticado; caso contrário, opera no modo público, suficiente para leitura de dados de mercado. O controle de taxa de requisições é gerenciado automaticamente pelo CCXT via `enableRateLimit=True`.

São expostas duas funções públicas. A função `buscar_dados_historicos(par, intervalo, qtd)` invoca `fetch_ohlcv()` e converte o resultado em um `pd.DataFrame` indexado por `timestamp` com colunas `Abertura`, `Maxima`, `Minima`, `Fechamento` e `Volume`, conforme o Algoritmo 2. A função `buscar_preco_atual(par)` consulta o endpoint `ticker` e retorna um dicionário com preço corrente, variação

percentual em 24h, máxima e mínima do dia e volume negociado.

Listing 2. Função de busca de dados históricos.

```
1 def buscar_dados_historicos(par, intervalo, qtd):
2     cliente = obter_cliente_binance()
3     dados = cliente.fetch_ohlcv(
4         par, intervalo, limit=qtd)
5     df = pd.DataFrame(dados, columns=[
6         "timestamp", "Abertura", "Maxima",
7         "Minima", "Fechamento", "Volume"])
8     df["timestamp"] = pd.to_datetime(
9         df["timestamp"], unit="ms")
10    df.set_index("timestamp", inplace=True)
11    return df
```

C. Módulo de Análise Técnica — *analise_tecnica.py*

Este módulo implementa todos os cálculos de indicadores técnicos utilizando exclusivamente a biblioteca Pandas, sem dependências externas especializadas em análise técnica. A decisão de implementar os indicadores do zero confere maior transparência e controle sobre os parâmetros de cálculo.

A função `calcular_media_movel(df, janela)` aplica `rolling(window).mean()` sobre a coluna `Fechamento`. A função `calcular_rsi(df, periodo=14)` computa as variações diárias com `diff()`, separa ganhos e perdas via `clip(lower=0)` e `clip(upper=0)`, calcula médias móveis simples de cada componente e obtém o RSI conforme a Equação 2. A função `calcular_macd()` utiliza o método `ewm(span, adjust=False).mean()` do Pandas para calcular as EMAs de 12 e 26 períodos e retorna um `DataFrame` com as colunas `macd`, `sinale_macd` e `histograma_macd`. Por fim, `calcular_bandas_bollinger(df, janela=20, desvios=2.0)` aplica `rolling(window).std()` para obter o desvio padrão e constrói as bandas conforme as Equações 4 e 5.

A função pública `enriquecer_dataframe(df)` agrega todas essas colunas ao `DataFrame` original, realizando uma cópia prévia para não modificar o dado de entrada. A função `resumir_indicadores(df)` extrai os valores da última linha (período mais recente) e os retorna em um dicionário com arredondamento e tratamento de valores ausentes via `pd.notna()`, conforme ilustrado a seguir:

Listing 3. Resumo dos indicadores da última vela.

```

1 def resumir_indicadores(df):
2     ultima = df.iloc[-1]
3     return {
4         "preco_fechamento": round(ultima["Fechamento"],
5                                     2),
6         "mm20": round(ultima["MM20"], 2) if pd.notna(
7             ultima["MM20"]) else None,
8         "mm50": round(ultima["MM50"], 2) if pd.notna(
9             ultima["MM50"]) else None,
10        "rsi14": round(ultima["RSI14"], 2) if pd.notna(
11            ultima["RSI14"]) else None,
12        "macd": round(ultima["MACD"], 4) if pd.notna(
13            ultima["MACD"]) else None,
14        "sinal_macd": round(ultima["Sinal_MACD"], 4)
15            if pd.notna(ultima["Sinal_MACD"]) else None,
16        "bollinger_superior": round(ultima["
17            Bollinger_Superior"], 2)
18            if pd.notna(ultima["Bollinger_Superior"])
19            else None,
20        "bollinger_inferior": round(ultima["
21            Bollinger_Inferior"], 2)
22            if pd.notna(ultima["Bollinger_Inferior"])
23            else None,
24    }

```

D. Módulo de Geração de Gráfico — *gerador_grafico.py*

O gráfico técnico é gerado com as bibliotecas *mplfinance* e *matplotlib*, operando em modo *headless* (*matplotlib.use("Agg")*), o que garante compatibilidade com ambientes sem interface gráfica, como servidores e contêineres. O módulo produz um gráfico composto por três painéis verticais com proporções (4, 1.5, 1.5):

- 1) **Painel de preço:** candlesticks com verde para candles de alta e vermelho para baixa, sobrepostos com as linhas MM20 (azul), MM50 (laranja), banda superior de Bollinger e banda inferior de Bollinger (linhas tracejadas em tom violeta), e banda central (tracejada cinza);
- 2) **Painel de volume:** barras verticais coloridas conforme a direção do candle;
- 3) **Painel RSI:** linha contínua do RSI-14 com linhas horizontais de referência tracejadas em 30 e 70.

O tema visual é escuro (#0D1117), consistente com o dashboard web. A imagem final é exportada em formato PNG com resolução de 150 DPI. Esse arquivo PNG é o artefato visual submetido ao Gemini na etapa seguinte, sendo a resolução suficiente para que o modelo interprete os padrões gráficos com fidelidade.

E. Módulo Analisador com IA — *analisador_ia.py*

Este módulo encapsula toda a lógica de comunicação com a API do Google Gemini e é composto por três funções:

_construir_prompt(indicadores): recebe o dicionário de indicadores e constrói um prompt textual estruturado que contextualiza o Gemini como um *trader* especialista em análise técnica. O prompt apresenta os valores numéricos dos indicadores acompanhados de interpretações qualitativas automáticas: o RSI é classificado como “sobrecomprado” ($RSI > 70$), “sobrevendido” ($RSI < 30$) ou “neutro”; o MACD indica se está acima ou abaixo da linha de sinal; as médias

móveis indicam se há tendência de alta ou baixa com base no cruzamento MM20/MM50. O modelo é instruído a responder **exclusivamente** em formato JSON com os campos *acao* (COMPRAR, VENDER ou MANTER), *justificativa*, *preco_alvo* e *tempo_estimado*.

_extrair_json_da_resposta(texto): lida com a variabilidade nas respostas do modelo, removendo marcações Markdown (blocos ``json) antes de realizar o *json.loads()*. Após a decodificação, valida o campo *acao* contra os valores permitidos; caso seja inválido, substitui por “MANTER”.

analizar_mercado_com_gemini(caminho_grafico, indicadores): função pública que instancia o cliente Gemini com a chave configurada em variável de ambiente, abre o PNG com a biblioteca Pillow e realiza a chamada multimodal ao modelo, conforme o Algoritmo 4:

Listing 4. Chamada multimodal ao modelo Gemini.

```

1 cliente = genai.Client(api_key=chave_api)
2 imagem = PIL.Image.open(caminho_grafico)
3 prompt = _construir_prompt(indicadores)
4
5 resposta = cliente.models.generate_content(
6     model="gemini-2.5-flash",
7     contents=[prompt, imagem]
8 )
9 resultado = _extrair_json_da_resposta(
10     resposta.text)

```

O envio simultâneo de prompt (texto) e imagem (objeto Pillow) na lista *contents* caracteriza a chamada multimodal. A função inclui blocos *try/except* para tratar *JSONDecodeError* e erros gerais de conexão com a API, retornando respostas de *fallback* estruturadas em ambos os casos.

F. Módulo do Dashboard — *gerador_dashboard.py*

O módulo de frontend gera um arquivo HTML completamente autossuficiente, todas as dependências JavaScript e CSS são embutidas no arquivo, que pode ser aberto diretamente em qualquer navegador sem servidor web. O dashboard é estruturado em cinco blocos visuais:

- 1) **Cabeçalho:** logotipo do sistema, par e timeframe analisados, e badge animado com horário da análise e ponto pulsante indicando status ao vivo;
- 2) **Grade de preços:** quatro cards com preço atual, variação percentual em 24h (com cor dinâmica por sinal), máxima, mínima e volume do dia;
- 3) **Gráfico Plotly interativo:** painel unificado com candlesticks, MM20, MM50, Bandas de Bollinger, barras de volume e RSI, com suporte a zoom, pan, hover com tooltip unificado e seletor de intervalos;
- 4) **Cards de indicadores:** exibe numericamente RSI, MACD, linha de sinal, MM20, MM50 e ambas as bandas de Bollinger, com interpretações qualitativas coloridas;
- 5) **Card de recomendação da IA:** exibe a ação (COMPRAR em verde, VENDER em vermelho, MANTER

em amarelo) com emoji correspondente, justificativa técnica completa em parágrafo, preço-alvo e estimativa de tempo.

O design adota o tema escuro da paleta do GitHub (#0D1117), tipografia Inter via Google Fonts, gradientes sutis e micro-animações CSS (*slide-in*, *pulse* no badge e *blink* nos indicadores de status), além de *media queries* para compatibilidade com dispositivos móveis.

V. RESULTADOS E DISCUSSÃO

A. Execução do Sistema

O CriptoIA foi testado com o par BTC/USDT (Bitcoin/Dólar Tether), variando os timeframes disponíveis (1h, 4h, 1d e 1w). O Bitcoin foi escolhido como ativo de referência por ser o de maior liquidez e volume de dados históricos disponíveis na API pública da Binance, oferecendo um cenário representativo para validação do sistema. A execução completa, desde a coleta de dados até a abertura do dashboard no navegador, tem duração média de 8 a 15 segundos em conexão banda larga. A maior parte desse tempo é consumida pela chamada à API do Gemini (4–10 s, dependendo da carga dos servidores do Google) e pela geração do gráfico PNG (1–2 s). A coleta de dados da Binance e a geração do HTML são praticamente instantâneas em comparação. Por ser parametrizável via linha de comando, o sistema suporta qualquer par listado na Binance, sendo a extensão para outros ativos uma das direções previstas para trabalhos futuros.

B. Qualidade da Análise Gerada pelo Gemini

O modelo Gemini 2.5 Flash demonstrou consistente capacidade de integrar as informações visuais do gráfico com os dados numéricos dos indicadores presentes no prompt. Nas análises realizadas, o modelo identificou corretamente: cruzamentos de médias móveis (*golden cross* e *death cross*); compressões e expansões das Bandas de Bollinger (*squeeze* e *breakout*); divergências entre preço e RSI; e estados de sobrecompra e sobrevenda. A justificativa gerada consistiu, em geral, em dois ou três parágrafos técnicos coerentes com o estado dos indicadores, com linguagem especializada e referências aos valores numéricos exatos. Um exemplo de resposta JSON gerada pelo modelo é apresentado na Listagem 5:

Listing 5. Exemplo de resposta JSON do Gemini.

```
1 {
2   "acao": "COMPRAR",
3   "justificativa": "O RSI de 42 indica saída de
4     zona de sobrevenda com recuperacao de momentum.
5     O MACD cruzou acima da linha de sinal,
6     confirmando a tendencia de alta. A MM20
7     ($94.200) cruzou acima da MM50 ($91.800),
8     configurando um golden cross. O preco esta
9     proximo a banda inferior de Bollinger,
10    indicando potencial de reversao.",
11   "preco_alvo": "$101.500,00",
12   "tempo_estimado": "7 a 14 dias"
13 }
```

C. Limitações Identificadas

Durante o desenvolvimento e os testes, foram observadas as seguintes limitações:

- **Caráter orientativo:** a recomendação é baseada exclusivamente em análise técnica e não considera fundamentos do ativo, notícias de mercado, regulações ou condições macroeconômicas [3]. O sistema deve ser utilizado como ferramenta de apoio, e não como conselho de investimento;
- **Consistência da IA:** em situações raras, o modelo retornou JSON sintaticamente malformatado ou com campo `acao` fora dos valores esperados; o módulo `_extrair_json_da_resposta()` trata esses casos com *fallback* automático para a ação “MANTER”;
- **Limite de dados históricos:** a API pública da Binance limita o retorno a 1.000 candles por requisição, o que pode ser insuficiente para análises de longo prazo com timeframes diários ou semanais;
- **Ausência de backtesting:** o sistema não inclui mecanismos de avaliação histórica das recomendações geradas, tornando impossível mensurar objetivamente a taxa de acerto das sugestões da IA;
- **Dependência de chave de API:** a etapa de análise com IA requer uma chave válida da API do Google Gemini, o que pode representar custo adicional em uso intensivo.

D. Comparação com Trabalhos Relacionados

Sistemas baseados em aprendizado de máquina supervisionado, como modelos LSTM para predição de séries temporais financeiras [15], exigem conjuntos de dados históricos rotulados, etapas de pré-processamento, treinamento e validação, e tendem a apresentar alto grau de ajuste ao período de treinamento. O CriptoIA não requer nenhuma dessas etapas: a análise é realizada em tempo real, com base nos dados mais recentes disponíveis.

A abordagem por *prompting* multimodal é mais flexível e diretamente interpretável, pois o modelo justifica em linguagem natural as razões para cada recomendação. Sistemas de análise técnica tradicionais, como os descritos em [16], geralmente produzem sinais binários (compra/venda) sem justificativa explicativa. O CriptoIA diferencia-se ao combinar o sinal com uma justificativa qualitativa e estimativa de horizonte temporal, aproximando a saída do sistema ao raciocínio de um analista humano.

VI. CONCLUSÃO

Este trabalho apresentou o CriptoIA, um sistema de análise de criptomoedas que integra indicadores técnicos clássicos com a capacidade multimodal do modelo Google Gemini 2.5 Flash. A arquitetura modular adotada permite a extensão individual de cada componente sem impacto nos demais: novos indicadores podem ser adicionados em `analise_tecnica.py`; outros modelos de IA (como GPT-4o ou Claude) podem ser integrados em `analizador_ia.py`; e o layout do

dashboard pode ser customizado independentemente em `gerador_dashboard.py`.

O sistema demonstrou funcionalidade consistente em cenários reais de mercado, gerando análises tecnicamente coerentes em tempo adequado para uso prático. A estratégia de *prompt engineering* com interpretações qualitativas automáticas dos indicadores mostrou-se eficaz para enriquecer o contexto enviado ao modelo, resultando em justificativas mais precisas e específicas. A integração multimodal, envio simultâneo de imagem de alta resolução e dados numéricos, representa uma abordagem inovadora que supera sistemas baseados exclusivamente em métricas textuais.

Como trabalhos futuros, destacam-se as seguintes direções:

- 1) Implementação de módulo de *backtesting* para avaliação histórica da precisão das recomendações geradas em diferentes condições de mercado;
- 2) Suporte a análise paralela de múltiplos pares e timeframes, com geração de relatórios comparativos;
- 3) Desenvolvimento de interface web com Flask ou FastAPI para facilitar o acesso a usuários não técnicos, com suporte a histórico de análises;
- 4) Avaliação comparativa sistemática das recomendações do sistema com modelos de aprendizado supervisionado (LSTM, Transformer) e estratégias *buy-and-hold*.

O código-fonte completo do sistema está disponível publicamente no GitHub, com documentação técnica detalhada e instruções de instalação via `pip`: <https://github.com/Teixeirx/tcc1>.

AGRADECIMENTOS

O autor agradece ao corpo docente do curso de Ciência da Computação da Universidade Franciscana e ao orientador pelo suporte e direcionamento durante o desenvolvimento deste trabalho.

REFERÊNCIAS

- [1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. [Online]. Disponível em: <https://bitcoin.org/bitcoin.pdf>
- [2] CoinMarketCap, "Global Cryptocurrency Market Cap Charts," 2024. [Online]. Disponível em: <https://coinmarketcap.com/charts/>
- [3] J. L. Pinheiro, *Mercado de Capitais: Fundamentos e Técnicas*, 8. ed. São Paulo: Atlas, 2019.
- [4] A. Assaf Neto, *Mercado Financeiro*, 11. ed. São Paulo: Atlas, 2012.
- [5] J. J. Murphy, *Technical Analysis of the Financial Markets*. Nova York: New York Institute of Finance, 1999.
- [6] F. Cavalcante, F. Misumi e R. Rudge, *Mercado de Capitais: O Que É, Como Funciona*, 7. ed. Rio de Janeiro: Elsevier, 2005.
- [7] Google DeepMind, "Gemini: A Family of Highly Capable Multimodal Models," *arXiv preprint arXiv:2312.11805*, 2024.
- [8] R. Maia e P. Gonçalves, "Inteligência Artificial Generativa Aplicada a Finanças: Uma Revisão Sistemática," *Revista Brasileira de Finanças*, vol. 22, no. 1, pp. 1–24, 2024.
- [9] R. D. Edwards, J. Magee e W. H. C. Bassetti, *Technical Analysis of Stock Trends*, 11. ed. Boca Raton: CRC Press, 2018.
- [10] J. W. Wilder, *New Concepts in Technical Trading Systems*. Greensboro: Trend Research, 1978.
- [11] J. Bollinger, *Bollinger on Bollinger Bands*. Nova York: McGraw-Hill, 2001.
- [12] H. Liu, C. Li, Q. Wu e Y. J. Lee, "Visual Instruction Tuning," in *Proc. NeurIPS*, 2023.
- [13] Binance, "Binance API Documentation," 2024. [Online]. Disponível em: <https://binance-docs.github.io/apidocs/>

- [14] CCXT, "CCXT — CryptoCurrency eXchange Trading Library," 2024. [Online]. Disponível em: <https://github.com/ccxt/ccxt>
- [15] T. Fischer e C. Krauss, "Deep learning with long short-term memory networks for financial market predictions," *European Journal of Operational Research*, vol. 270, no. 2, pp. 654–669, 2018.
- [16] R. C. Cavalcante *et al.*, "Computational Intelligence and Financial Markets: A Survey and Future Directions," *Expert Systems with Applications*, vol. 55, pp. 194–211, 2016.